



Functions, Structures and Enums

Functions, parameters, prototypes, callbacks, structures, enumerations, and embedded practice

Embedded Bare-Metal Diploma

DAY 3

6 Hours

```
typedef void (*Action)
(void);

struct Device { int id; };
enum State { OFF, ON };
```

Prepared by Mahmoud Morsy

NTI | National Telecommunication Institute



One day connects reusable functions with structured data

Session roadmap

00:00

01

**Functions and
prototypes**

01:10

02

**Parameters
and
arguments**

02:20

03

**Pointer
parameters
and callbacks**

03:20

04

**Structures
and
member
access**

04:20

05

**Enums,
integration
and
assessment**

Teaching rhythm: explain -> demonstrate -> 5-10 minute task -> review



By the end, students can organize data and behavior in C

Learning outcomes

- Distinguish function declarations, definitions and calls.
- Separate parameters from arguments.
- Pass values or addresses intentionally.
- Declare, assign and call compatible function pointers.
- Define, initialize and access structures.
- Model controller states with enums and integrate all topics.

TODAY'S OUTPUT

7

**working examples
and completed
exercises**



Functions divide embedded programs into testable tasks

Why functions matter

01

One responsibility

A short function performs one clear task and is easier to verify.

02

Reuse and testing

Call the same behavior from several places without copying code.

03

Driver structure

Initialization, reading and control become separate driver functions.

Embedded pattern: main() coordinates the application; small functions configure, read and act.



A function has a return type, name, parameters and body

Function anatomy

```
float calculatePower(float voltage, float current)
{
    return voltage * current;
}

int main(void)
{
    float p = calculatePower(12.0f, 1.5f);
    return 0;
}
```

Return type

float

Name

calculatePower

Parameters

**float voltage, float
current**

Return value

voltage * current

A declaration describes the
interface; the definition
supplies the body.



A prototype lets C check a call before its definition

Declaration and definition

```
float power(float v, float i);    //  
declaration  
  
int main(void)  
{  
    float p = power(12.0f, 1.5f); // call  
}  
  
float power(float v, float i)    //  
definition  
{  
    return v * i;  
}
```

READ IT TOP TO BOTTOM

- **Prototype:** ends with a semicolon.
- **Call:** supplies arguments.
- **Definition:** contains the function body.
- **Types must be compatible.**
- **Place prototypes before the first call.**

Declare before use



Every function call follows the same execution path

Call sequence



Execution flow

After return, execution continues at the statement following the function call.



Exercise: identify declaration, definition, call and return

6-minute practice

1 `int add(int a, int b);`

2 `int result = add(4, 6);`

3 `int add(int a, int b) { return a + b; }`

4 `return a + b;`

Work individually - 6 minutes



Answer: each function form has a different role

Instructor review

```
int add(int a, int b);           //  
declaration  
int result = add(4, 6);         // call  
  
int add(int a, int b)           //  
definition  
{  
    return a + b;
```

ASK WHY

**Why does the
prototype
end with a
semicolon?
Why are 4 and 6
arguments, not
parameters?**



Parameters receive data; arguments supply the values

Parameters and arguments

```
int add(int first, int second)
{
    return first + second;
}

int result = add(4, 6);
```

COMPARE THE TERMS

Term	Meaning
Parameter	Name in a declaration or definition
Argument	Value or expression supplied in a call
first	Parameter inside add()
4	Argument in add(4, 6)

The call copies argument values into the function's parameter variables.



Declaration, definition and call serve different jobs

Function terminology

Stage	Recognize it	Purpose
Declaration	Prototype + semicolon	Announce the interface
Definition	Body inside braces	Implement the function
Call	name(arguments)	Execute the function
Parameter	Name in header	Receive data
Argument	Value in call	Supply data

Compatibility rule: argument count and types must match the function prototype.



The return type defines what the function sends back

Return types

Return type	Typical use	Example
void	No returned value	<code>void stopMotor(void);</code>
int	Integer status	<code>int readSwitch(void);</code>
float	Measurement	<code>float readVoltage(void);</code>
enum State	Named state	<code>enum State getState(void);</code>
struct Sensor	Grouped record	<code>struct Sensor readSensor(void);</code>

A non-void function must return a compatible value on every reachable path.



C passes ordinary arguments by value

Pass by value

```
void setSpeed(int speed)
{
    speed = 100;
}

int main(void)
{
    int motorSpeed = 20;
    setSpeed(motorSpeed);
    printf("%d", motorSpeed); // 20
}
```

ARGUMENT
COPY

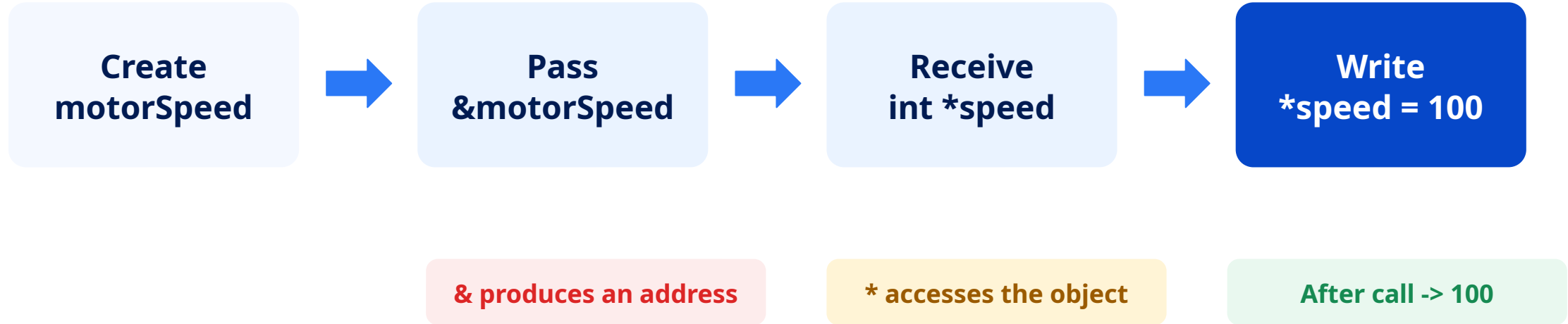
LOCAL CHANGE

ORIGINAL
UNCHANGED



An address lets a function modify the caller's variable

Pointer parameters



Use a pointer parameter when a function must read or modify the caller's object.



Dereferencing changes the original object through its address

Pointer parameter trace

```
void setSpeed(int *speed)
{
    *speed = 100;
}

int main(void)
{
    int motorSpeed = 20;
    setSpeed(&motorSpeed);
}
```

TRACE THE CALL

Before **20**

Address **&motorSpeed**

After **100**



Exercise: write a function that swaps two integers

10-minute practice

Item	Value
Before	$x = 3, y = 7$
Call	<code>swap(&x, &y)</code>
After	$x = 7, y = 3$

REQUIREMENTS

**`void swap(int*,
int*)`**

Use `temp + *a + *b`

Work individually - 10 minutes



Answer: addresses let swap update both caller variables

Instructor review

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int x = 3;
int y = 7;
swap(&x, &y);
```

TRACE VALUES

temp = 3

***a = 7**

***b = 3**

Result: x = 7, y = 3



A function pointer stores a compatible code address

Function pointers

```
void startMotor(void) { /* hardware ON */ }  
void stopMotor(void) { /* hardware OFF */ }  
  
void (*action)(void);  
  
action = startMotor;  
action();
```

READ THE DECLARATION

- void: return type
- (*action): pointer variable
- (void): no parameters
- Assigned functions must match
- Calling action() runs the selection

Function pointers enable callbacks and selectable operations.



Declare, assign and call a function pointer in three steps

Function pointer workflow

DECLARE

Pointer type

```
void (*action)(void);
```

ASSIGN

Select function

```
action = startMotor;
```

CALL

Run selection

```
action();
```

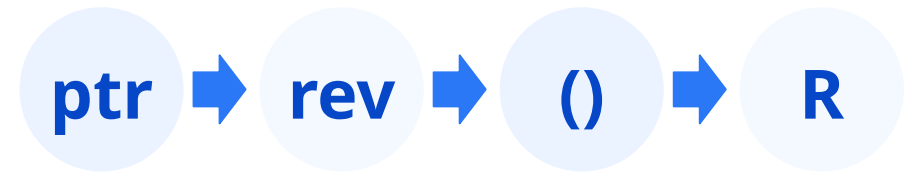
Return type and parameter types must match the selected function.



Trace which callback runs at execution time

5-minute quiz

```
void forward(void) { printf("F"); }  
void reverse(void) { printf("R"); }  
  
void (*move)(void);  
move = reverse;  
move();
```



Answer: reverse() runs -> R

Which function runs? What is printed?



Exercise: select one arithmetic operation with a pointer

10-minute practice

operation(8, 2)

**Do not call add() or subtract()
directly in the final line.**

- Write add() and subtract() functions.
- Declare int (*operation)(int, int).
- Read choice 1 or 2.
- Assign the matching function.
- Call the pointer and print the result.

Work individually - 10 minutes



Answer: one pointer selects either compatible operation

Instructor review

```
int add(int a, int b) { return a + b; }  
int subtract(int a, int b) { return a - b; }  
  
int (*operation)(int, int);  
operation = (choice == 1) ? add : subtract;  
  
printf("%d", operation(8, 2));
```

TRACE

choice = 1 -> add ->

10

choice = 2 ->

subtract -> 6

One interface, two
operations



typedef makes callback declarations easier to read

Function pointer aliases

```
typedef int (*Operation)(int, int);  
  
int calculate(Operation op, int a, int b)  
{  
    return op(a, b);  
}
```

Alias

Operation

Points to

int (*)(int, int)

Parameter

Operation op

Call

op(a, b)

**typedef changes the
spelling, not the function-
pointer type.**



Exercise: pass a callback into another function

10-minute practice

```
void execute(void (*command)(void));
```

Test: execute(ledOn) -> LED ON

- Create ledOn() and ledOff().
- Make execute() receive a callback.
- Call the selected operation inside execute().
- Test both commands from main().

Maximum time - 10 minutes



Answer: execute() runs whichever callback it receives

Instructor review

```
void ledOn(void) { printf("LED ON"); }  
void ledOff(void) { printf("LED OFF"); }  
  
void execute(void (*command)(void))  
{  
    command();  
}  
  
int main(void)  
{  
    execute(ledOn);  
    execute(ledOff);  
}
```



callback output



A structure groups related values into one record

Why structures matter

01

One device

An ID, a measurement and a state travel together as one object.

02

One type

A struct defines the same data layout for many device objects.

03

Clear interfaces

Functions can receive or return one complete device record.

Embedded use: package sensor measurements, configuration, limits and state.



A structure definition creates a custom data layout

Structure syntax

Element	Purpose	Example
Tag	Names the type	struct Sensor
Member	Stores an ID	int id;
Member	Stores a measurement	float temperature;
Variable	Allocates one object	struct Sensor s1;
typedef	Creates an alias	typedef struct Sensor Sensor;

The definition describes the layout; a variable creates storage for one object.



Use dot notation to access members of a structure

Structure objects

```
struct Sensor {  
    int id;  
    float temperature;  
    int active;  
};  
  
struct Sensor s1 = {1, 36.5f, 1};  
s1.temperature = 37.2f;
```

READ IT TOP TO BOTTOM

- Definition groups the members.
- s1 creates one object.
- The initializer follows member order.
- The dot operator selects a member.
- Access example: s1.temperature

One object, related data



Use -> when a pointer refers to a structure

Pointers to structures

```
void update(struct Sensor *sensor)
{
    sensor->temperature = 38.0f;
    sensor->active = 1;
}

int main(void)
{
    struct Sensor s1 = {1, 0.0f, 0};
    update(&s1);
}
```

&s1: address

sensor: pointer

**sensor-
>member: access**



An enum gives readable names to integer states

Enumeration types

DEFINE

Named state set

```
enum MotorState  
{ STOPPED, RUNNING,  
  FAULT };
```

DECLARE

Create variable

```
enum MotorState state;
```

ASSIGN

Select state

```
state = RUNNING;
```

By default, enumerator values begin at 0 and increase by 1.



Enums make switch statements easier to read

enum with switch

```
switch (state)
{
    case STOPPED:
        stopMotor();
        break;
    case RUNNING:
        startMotor();
        break;
    case FAULT:
        alarmOn();
        break;
    default:
        safeShutdown();
}
```

WHEN TO USE IT

- A finite set of named states
- Readable switch cases
- No unexplained magic numbers
- Prefixes can prevent name collisions

Use explicit values when a hardware protocol requires fixed numeric codes.



Final exercise: model a pump controller

15-minute integrated practice

Component	Requirement
enum	PUMP_OFF, PUMP_RUNNING, PUMP_FAULT
struct	id, pressure, state
callback	void (*action)(void)

REQUIREMENTS

**Create struct
Pump**

Select a callback

Update state

Print ID + state

Work independently - 15 minutes



Solution: combine enum, struct and a callback

Application review

```
enum PumpState { PUMP_OFF, PUMP_RUNNING,  
PUMP_FAULT };  
  
struct Pump  
{  
    int id;  
    float pressure;  
    enum PumpState state;  
};  
  
void startPump(void) { printf("Pump ON"); }  
void stopPump(void) { printf("Pump OFF"); }
```

```
struct Pump p = {1, 2.4f,  
PUMP_OFF};  
void (*action)(void) =  
startPump;  
action();  
p.state = PUMP_RUNNING;
```

Pump 1
2.4 bar
RUNNING



Teaching references

Sources

CORE C REFERENCE

GNU C Language Manual - functions, parameters, pointers, structures and enumerations

EMBEDDED COMPILER REFERENCE

Microchip MPLAB XC8 C Compiler User's Guide for AVR MCU

All examples, exercises, code solutions and explanatory layouts in this deck are original course materials. The official references were used to verify terminology and C syntax.

Links are available in the PowerPoint speaker notes.